Machine Learning

Variational Autoencoder

Dong-Kyu Chae

PI of the Data Intelligence Lab. @HYU

Dept. of Computer and Software

Hanyang University

Variational Autoencoder

- **ELBO:** $\mathcal{L}(\theta, \phi)$

$$\mathcal{L}(\theta, \phi) = \mathrm{E}_z[\log p_{\boldsymbol{\theta}}(x|z)] - \mathrm{E}_z\left[\log \frac{q_\phi(z|x)}{p(z)}\right]$$

$$= \mathrm{E}_z[\log p_{\boldsymbol{\theta}}(x|z)] - D_{KL}(q_\phi(z|x)||p(z))$$

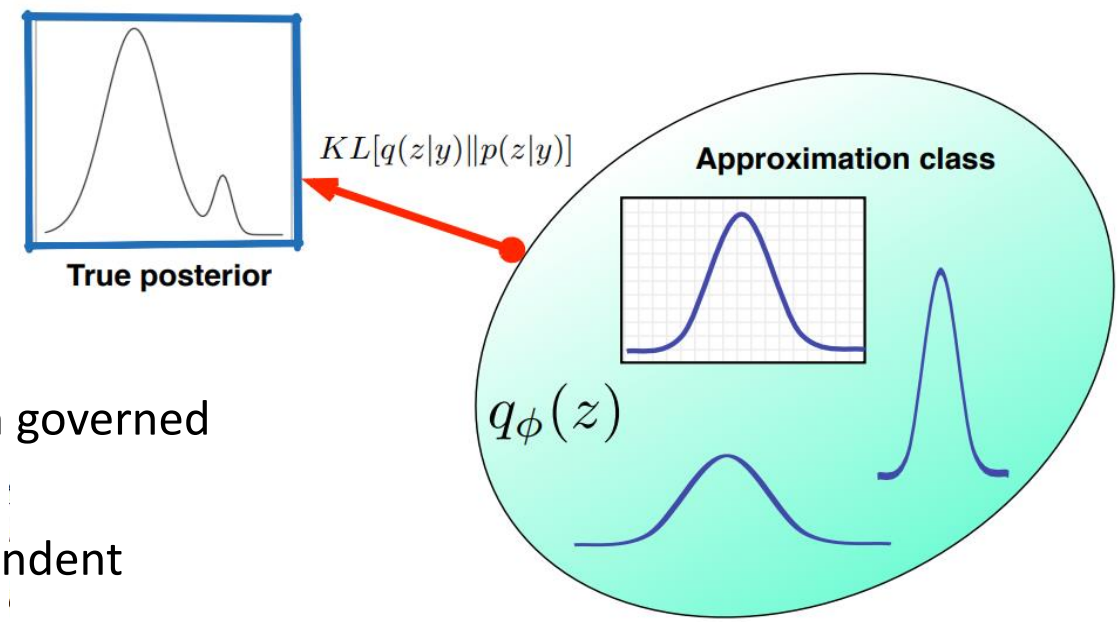
- $\log p(x) = \mathcal{L}(\theta, \phi) + D_{KL}(q_\phi(z|x)||\, p(z|x))$ ← This term lies in the context of variational inference!
  - Place high prob mass on many z values that could have generated x

- $\log p(x) \geq \mathrm{E}_z[\log P_{\boldsymbol{\theta}}(x|z)] - D_{KL}(q_\phi(z|x)||p(z))$

Background: Variational Inference

- **Goal: to approximate the exact posterior $p(z|x)$ with a simpler distribution $q(z)$, through an optimization manner**
  1. Posit $\mathcal{Q}$, a family of approximate <u>densities</u> over z
     - each $q(z) \in \mathcal{Q}$: a candidate approximation to the exact posterior
  2. Find the member of $\mathcal{Q}$ that minimizes D_{KL} to the exact posterior (optimization problem!)

$$q^*(z) = \arg\min_{q(z)\in\mathcal{Q}} D_{KL}[q(z)||p(z|x)]$$



  3. Common restriction on $\mathcal{Q}$
     - Use a parametric distribution governed by parameters $\phi \Rightarrow q_\phi(z)$
     - Each latent variable is independent

Background: Variational Inference

- **However, the objective function is not computable**

$$q^*(z) = \arg\min_{q(z)\in\mathcal{Q}} D_{KL}[q(z)||p(z|x)] \quad (1)$$

- **Reason:**

$$\begin{aligned} D_{KL}[q(z)||p(z|x)] &= \mathbb{E}_z[\log q(z)] - \mathbb{E}_z[\log p(z|x)] \\ &= \mathbb{E}_z[\log q(z)] - \mathbb{E}_z[\log p(z,x)] + \underbrace{\log p(x)}_{\text{intractable}} \end{aligned} \quad (2)$$

- **Solution:**
  - Optimize an alternative objective called ***evidence lower bound (ELBO)***

Background: Variational Inference

- **Rearranging (2) gives**

$$\underbrace{\log p(x)}_{\text{A constant wrt } q(z)} - D_{KL}[q(z)||p(z|x)] = \underbrace{\mathbb{E}_z[\log p(z,x)] - \mathbb{E}_z[\log q(z)]}_{\text{“ELBO”}} \quad (3)$$

- **Maximizing ELBO = minimizing the D_{KL}**

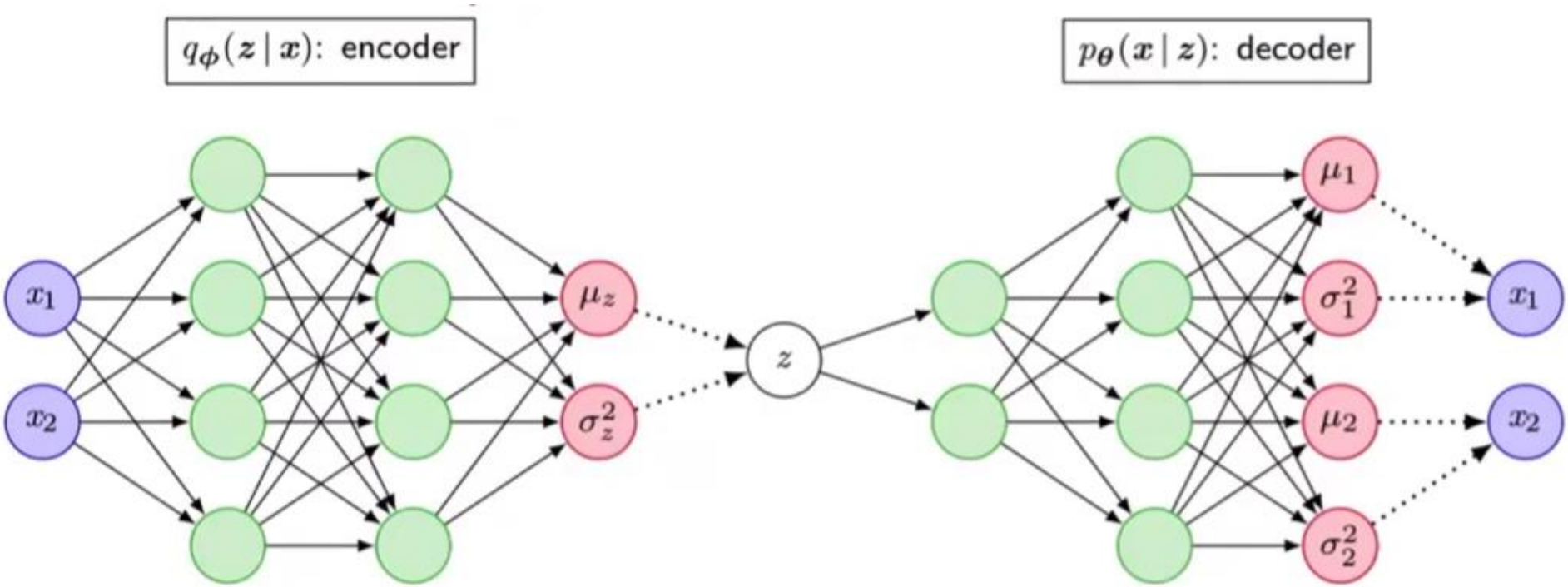
$$\begin{aligned}\mathcal{L}(q) &= \mathbb{E}_z[\log p(z,x)] - \mathbb{E}_z[\log q(z)] \\ &= \mathbb{E}_z[\log p(z)] + \mathbb{E}_z[\log p(x|z)] - \mathbb{E}_z[\log q(z)] \\ &= \underbrace{\mathbb{E}_z[\log p(x|z)]}_{\text{Expected log likelihood of the data}} - \underbrace{D_{KL}[q(z)||p(z)]}_{D_{KL} \text{ between prior } p(z) \text{ and } q(z)}\end{aligned}$$

- **Comparing with VAE ELBO**

$$\mathbb{E}_{z\sim q(z|x)}[\log p(x|z)] - D_{KL}[q(z|x)||p(z)]$$

VAE Architecture

- **Encoder and decoder are stacked**



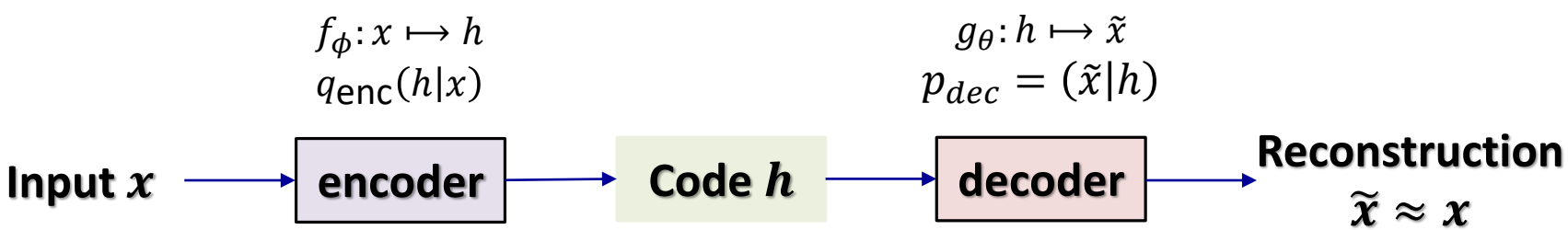
- **We can learn parameters ϕ and θ via backprop**
  - (spoiler) but, we need reparameterization trick

Background: Autoencoder (AE)

- **Neural net trained to *reconstruct* input x as output x (i.e., $x \approx \tilde{x}$)**
  - Learns code (= efficient representation of input data)
  - Unsupervised (no label)

- **Architecture**

	Function form	Parameterized by
Encoder	$f_\phi(x) = h$	ϕ
decoder	$g_\theta(h) = \tilde{x}$	θ



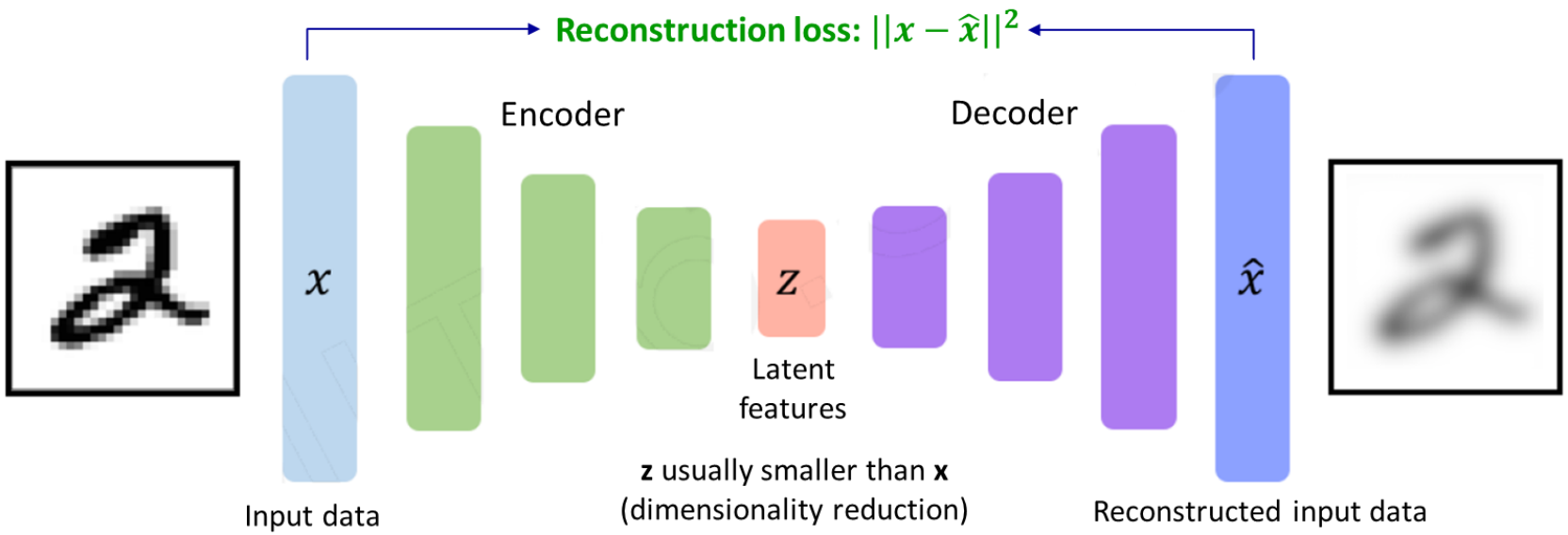
  - $\dim(x) > \dim(h)$: ***undercomplete*** $\Rightarrow$ capture important features of unput

Background: Autoencoder (AE)

Training

$$\phi^*, \theta^* = \arg\min_{\phi,\theta} L(x, \tilde{x})$$

- L: loss (e.g., mean squared error)



Applications

- Dimensionality reduction
- Feature learning

Training of VAE

- **Perform SGD to maximize the objective:**

$$\mathcal{L}(\theta,\phi) = \underbrace{-D_{KL}[q_\phi(z|x)||p(z)]}_{\text{1. encoding objective}} + \underbrace{\mathbb{E}_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{2. decoding objective}}$$

1. **Optimizing 1.: encoding objective $-\boldsymbol{D_{KL}[q_\phi(z|x)||p(z)]}$**
   - $\boldsymbol{D_{KL}}$ between two multivariate Gaussians can be computed in closed form
   - In general (K: dimensionality of variable)

$$D_{KL}[\mathcal{N}(\mu_0,\Sigma_0)||\mathcal{N}(\mu_1,\Sigma_1)]$$
$$= \frac{1}{2}\left(\mathrm{tr}(\Sigma_1^{-1}\Sigma_0) + (\mu_1-\mu_0)^\top\Sigma_1^{-1}(\mu_1-\mu_0) - K + \mathrm{In}\left(\frac{\det\Sigma_1}{\det\Sigma_0}\right)\right)$$

Training of VAE

- **In VAE, this simplifies to**

$$
\begin{aligned}
&D_{KL}[\mathcal{N}\big(\mu(x), \Sigma(x)\big) \,||\, \mathcal{N}(0, \mathrm{I})] \\
&= \frac{1}{2}\big(\mathrm{tr}\big(\Sigma(x)\big) + \mu(x)^{\top}\mu(x) - K - \mathrm{In}\det\Sigma(x)\big) \\
&= \frac{1}{2}\left(\sum_{k=1}^{K}\sigma_k^2(x) + \sum_{k=1}^{K}\mu_k^2(x) - \sum_{k=1}^{K}1 - \mathrm{In}\prod_{k=1}^{K}\sigma_k^2(x)\right) \\
&= \frac{1}{2}\sum_{k=1}^{K}\left(\sigma_k^2(x;\phi) + \mu_k^2(x;\phi) - 1 - \mathrm{In}\sigma_k^2(x;\phi)\right) = L_{enc}(\phi)
\end{aligned}
$$

* tr: trace det: determinant

- **Differentiable $\Rightarrow$ $\boldsymbol{L_{enc}(\phi)}$ can be minimized via SGD**

Training of VAE

Optimizing 2.: decoding objective $\mathbb{E}_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)]$

- Use Monte Carlo sampling: for each input $x^{(i)}$
  1. $z^{(i,l)}$ is sampled from encoder $q_\phi\left(z\middle|x^{(i)}\right)$
  2. $z^{(i,l)}$is fed to decoder ⇒ gives $\log p_\theta\left(x^{(i)}\middle|z^{(i,l)}\right)$
  3. repeat above steps L times to approximate

$$\mathbb{E}_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)] \approx \frac{1}{L}\sum_{l=1}^{L}\log p_\theta\left(x^{(i)}\middle|z^{(i,l)}\right)$$

- In practice, often $L = 1$

$$\Rightarrow \log p_\theta\left(x^{(i)}\middle|z^{(i)}\right)$$

- Recall that we assume **independence** between features

$$\Rightarrow \log p_\theta\left(x^{(i)}\middle|z^{(i)}\right) = \log\prod_{d=1}^{D} p_\theta\left(x_d^{(i)}\middle|z^{(i)}\right) = \sum_{d=1}^{D}\log p_\theta\left(x_d^{(i)}\middle|z^{(i)}\right)$$

Reparameterization Trick

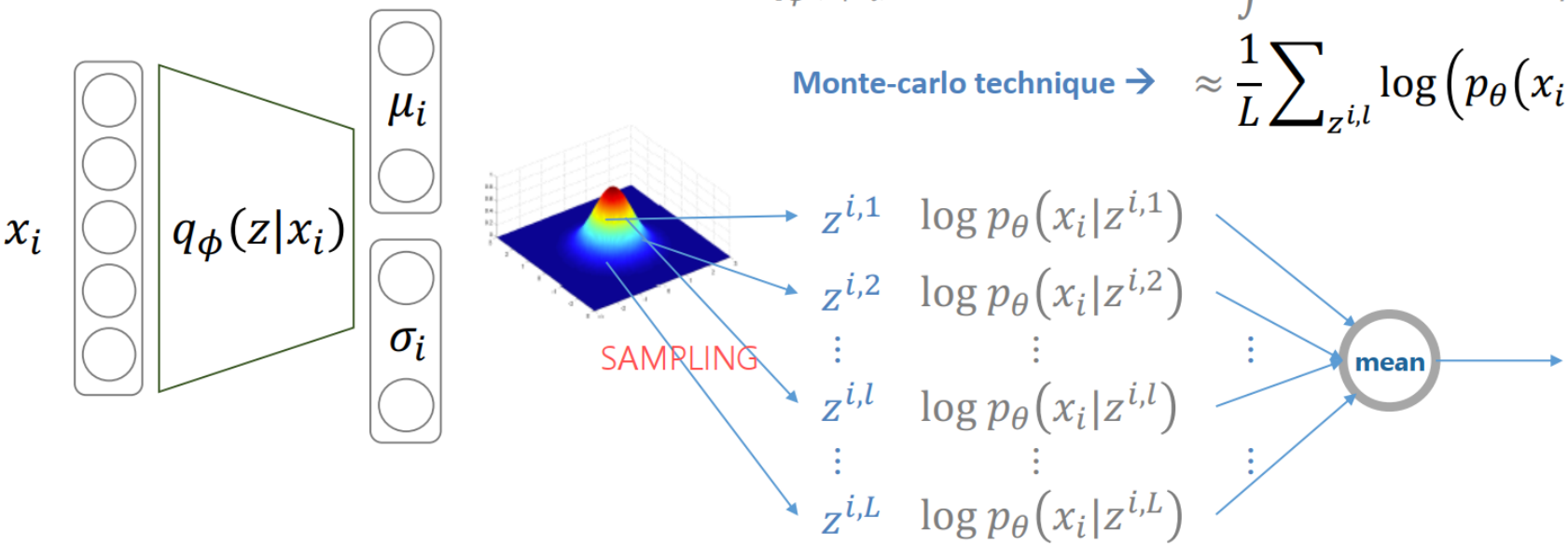
- **However, we cannot backprop gradient through the sampling process**
  - Sampling: not continuous ⇒ not differentiable
  - We need to trick called **reparameterization**
- **Idea: move the sampling process to the <u>input layer</u>**
  - Given $\boldsymbol{\mu}(\boldsymbol{x})$ and $\sum(\boldsymbol{x})$ (*i.e.*, mean and covariance of $\boldsymbol{q}(\boldsymbol{z}|\boldsymbol{x})$),
  - Instead of sampling $z^{(i,l)} \sim \mathcal{N}\left(\mu\left(x^{(i)}\right),\ \Sigma\left(x^{(i)}\right)\right)$ directly,
  - We can derive $z^{(i,l)}$ in the same way by
    1. First sampling $\epsilon^{(l)} \sim \mathcal{N}(0, \mathrm{I})$
    2. Then computing
    $$z^{(i,l)} = \mu\left(x^{(i)}\right) + \sigma(x^{(i)}) \odot \epsilon^{(l)}$$
    - $\odot$: element-wise multiplication
  - $z^{(i,l)}$ has the same distribution as before, but now we can do backprop

Reparameterization Trick

☐ Graphical view (before reparameterization trick)

$$\mathbb{E}_{q_\phi(z|x_i)}[\log(p_\theta(x_i|z))] = \int \log(p_\theta(x_i|z))q_\phi(z|x_i)dz$$

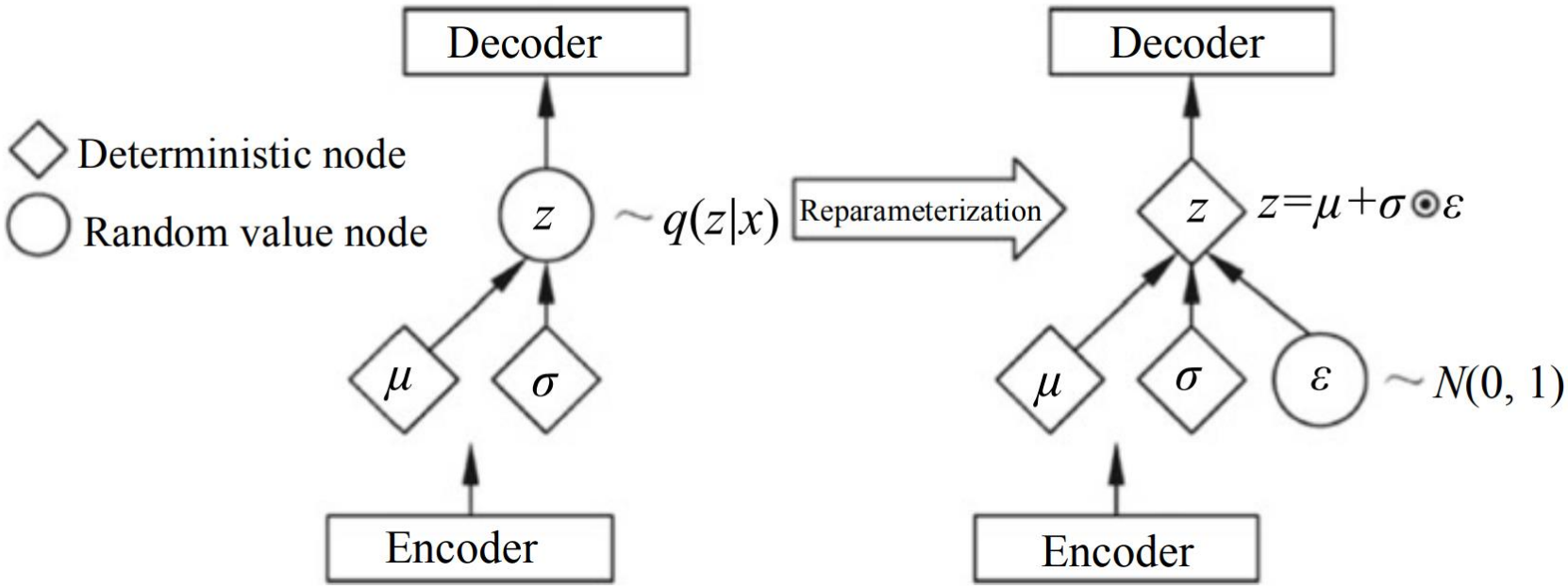
Monte-carlo technique → $$\approx \frac{1}{L}\sum_{z^{i,l}} \log\left(p_\theta(x_i|z^{i,l})\right)$$



- L is the number of samples for latent vector
- Usually L is set to 1 for convenience

Reparameterization Trick

Graphical view (reparameterization trick)



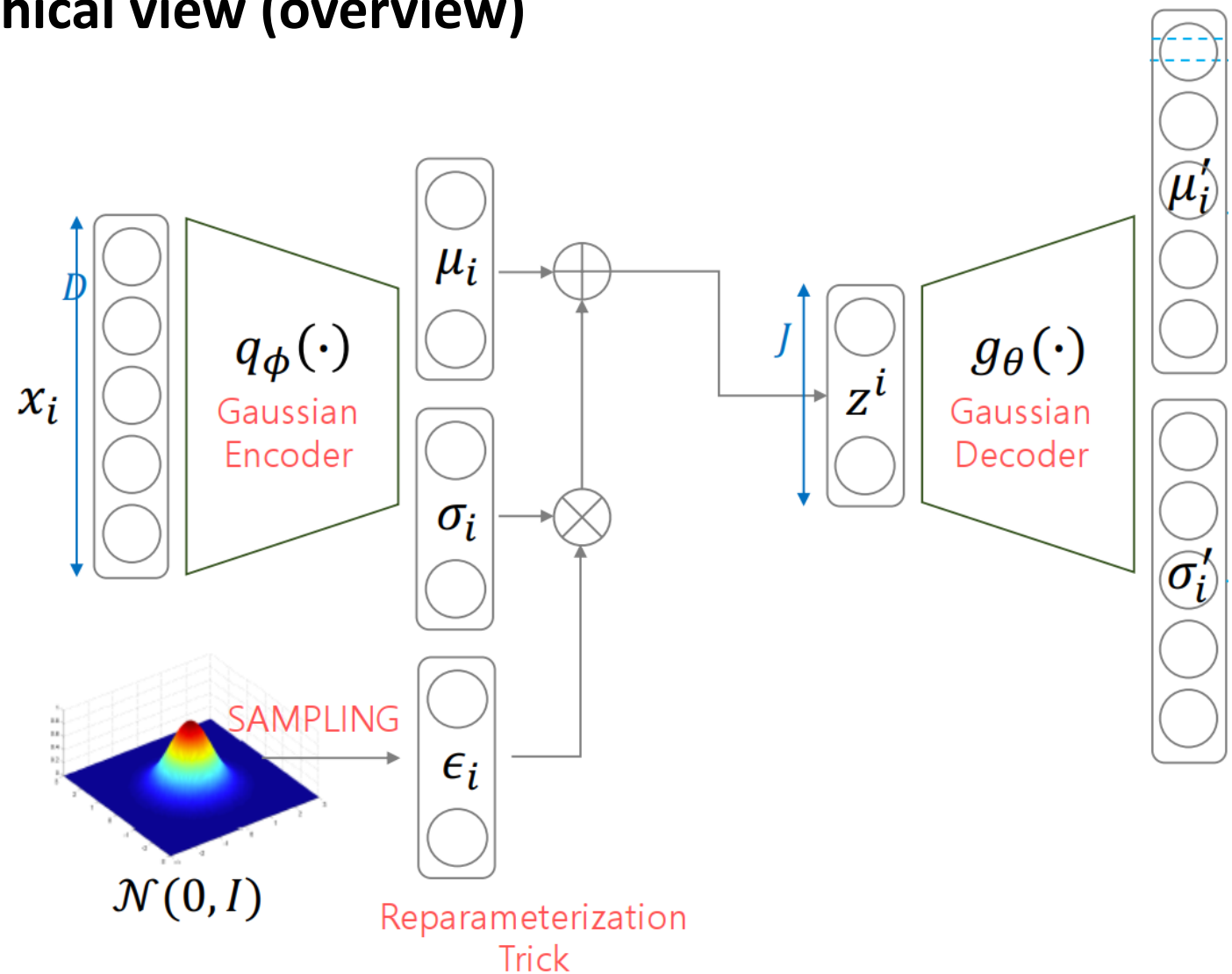
Sampling Process

$$z^{i,l} \sim N(\mu_i, \sigma_i^2 I) \Rightarrow z^{i,l} = \mu_i + \sigma_i \odot \epsilon$$

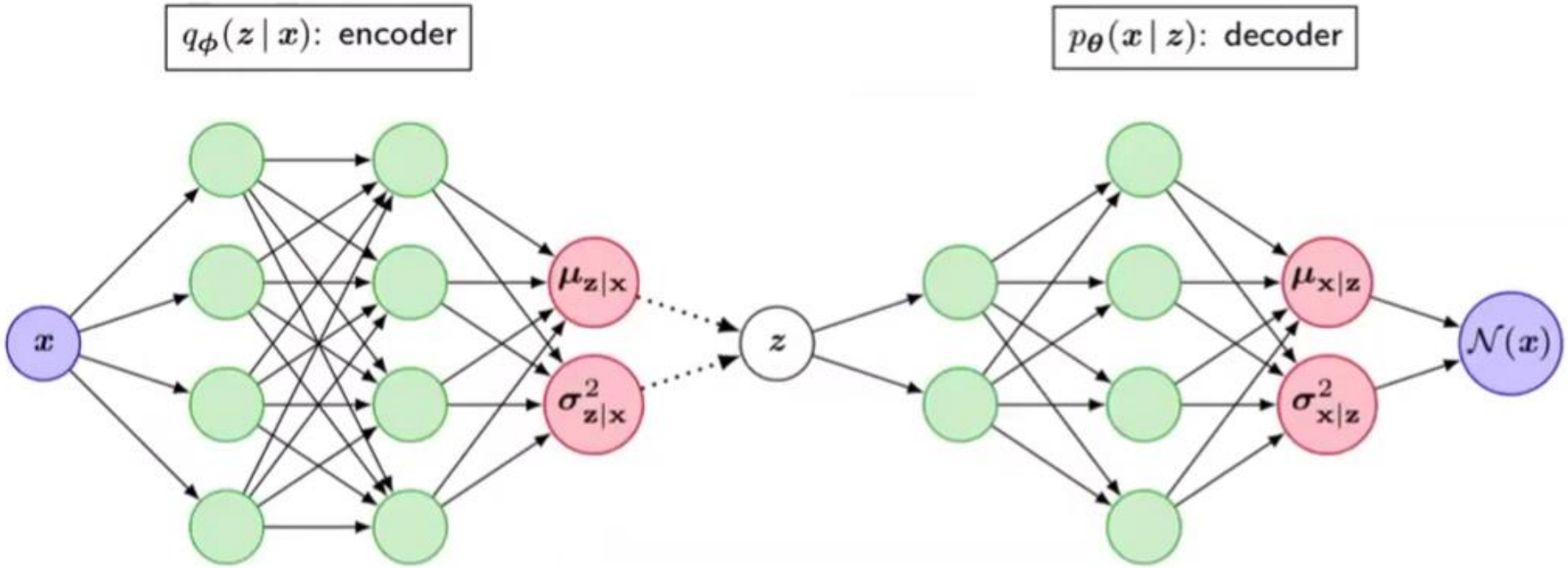
$$\epsilon \sim N(0, I)$$

Reparameterization Trick

☐ Graphical view (overview)



Training Overview



$$\mathcal{L}(\theta,\phi) = -D_{KL}[q_\phi(z|x)||p(x)] + \mathbb{E}_{z\sim q_{\phi(Z|X)}}[\log p_\theta(x|z)]$$ (when using SGD with L=1)

$$= \frac{1}{2}\sum_{k=1}^{K}\left(1 + \mathrm{In}\sigma_k^2(x^{(i)};\phi) - \mu_k^2(x^{(i)};\phi) - \sigma_k^2(x^{(i)};\phi)\right) + \log p_\theta\left(x^{(i)}\middle|z^{(i)}\right)$$

where $\log p_\theta\left(x^{(i)}\middle|z^{(i)}\right) = \log \mathcal{N}\left(x^{(i)};\mu\left(z^{(i)}\right),\sigma^2\left(z^{(i)}\right)I\right)$

$$\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

$$= \Sigma_d \frac{-1}{2}\log\left(2\pi\sigma_d^2\left(z^{(i)}\right)\right) - \frac{\left(x_d^{(i)} - \mu_d\left(z^{(i)}\right)\right)^2}{2\sigma_d^2(z^{(i)})}$$

where $z^{(i)} = \mu\left(x^{(i)}\right) + \sigma\left(x^{(i)}\right)\odot\epsilon^{(l)}$, $\epsilon^{(i)}\sim\mathcal{N}(0,\mathrm{I})$

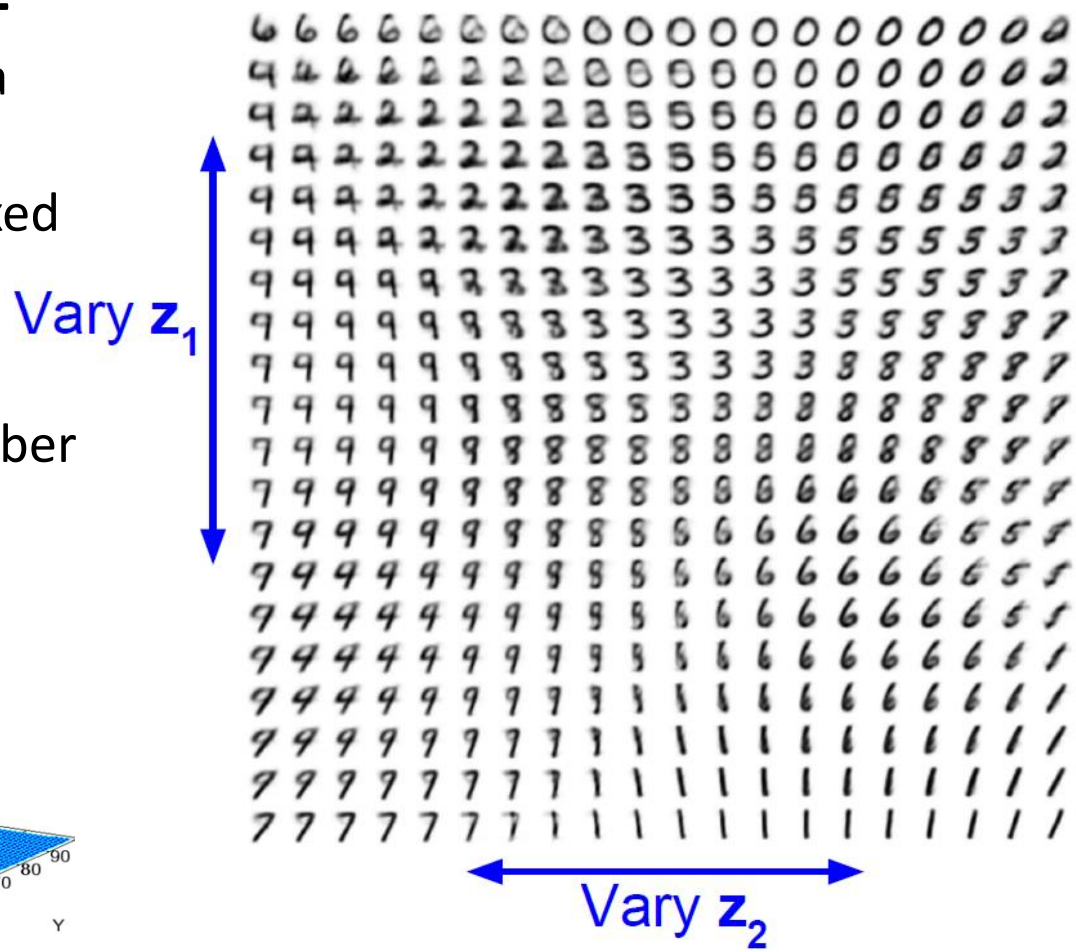
Sample Generation after Training

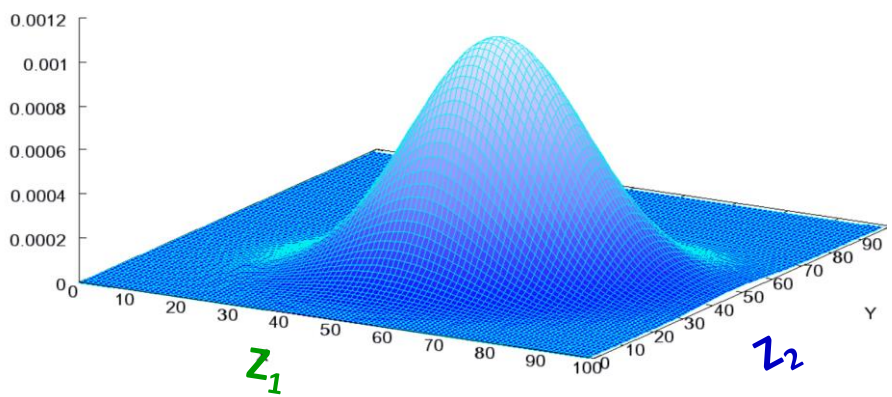
- **When we want to generate a new sample**
  - Simply input a sample $z \sim \mathcal{N}(0, \mathrm{I})$ into decoder
    - Do not use encoder at test time
  - Decoder will produce parameters of $p(x|z)$
  - We then sample a new x from $p(x|z)$

http://dpkingma.com/sgvb_mnist_demo/demo.html

Sample Generation after Training

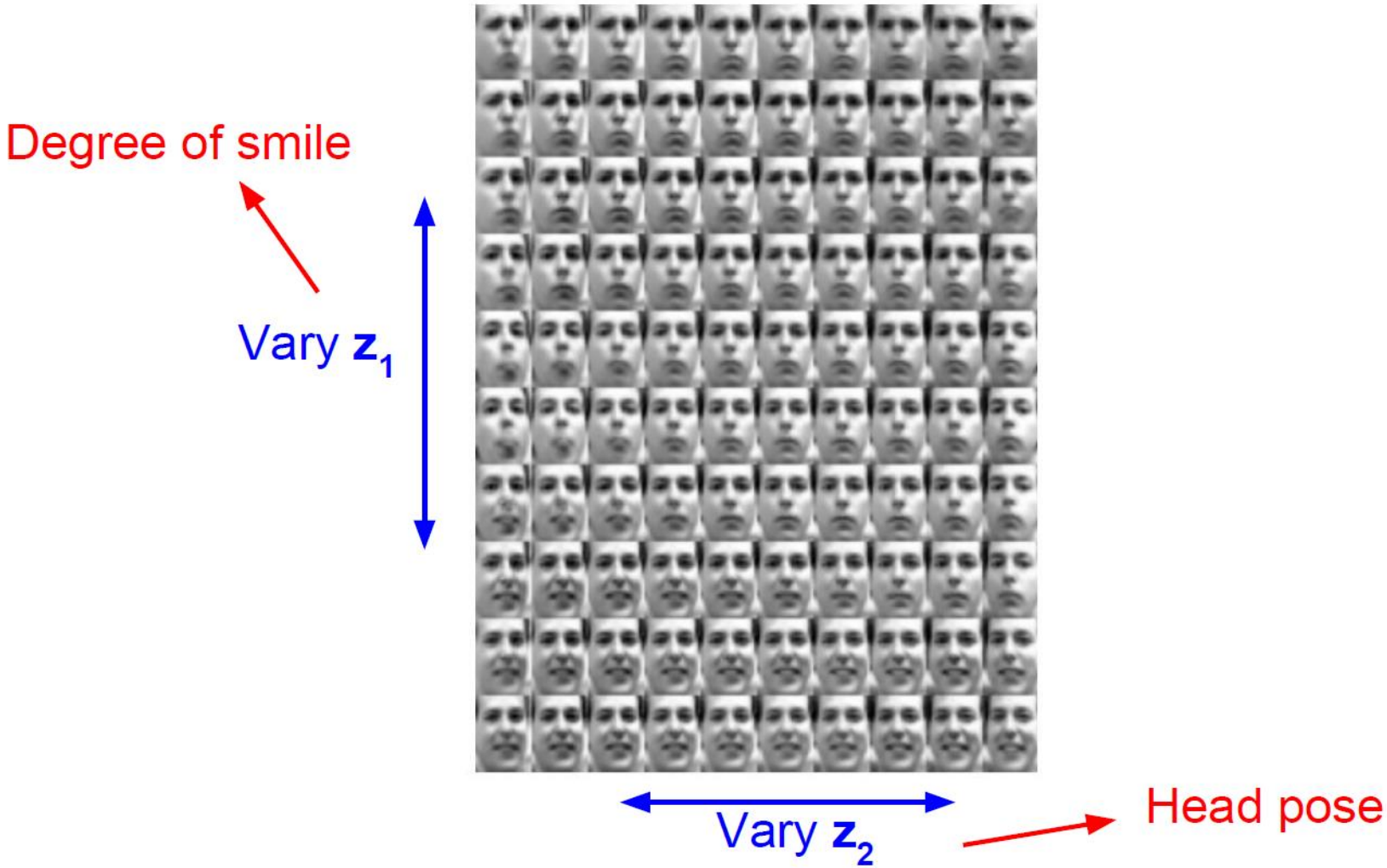
- **Generated samples w.r.t z**
  - Slowly increase or decrease a single latent variable while keeping all other variables fixed
  - Each number is smoothly transitioning to another number





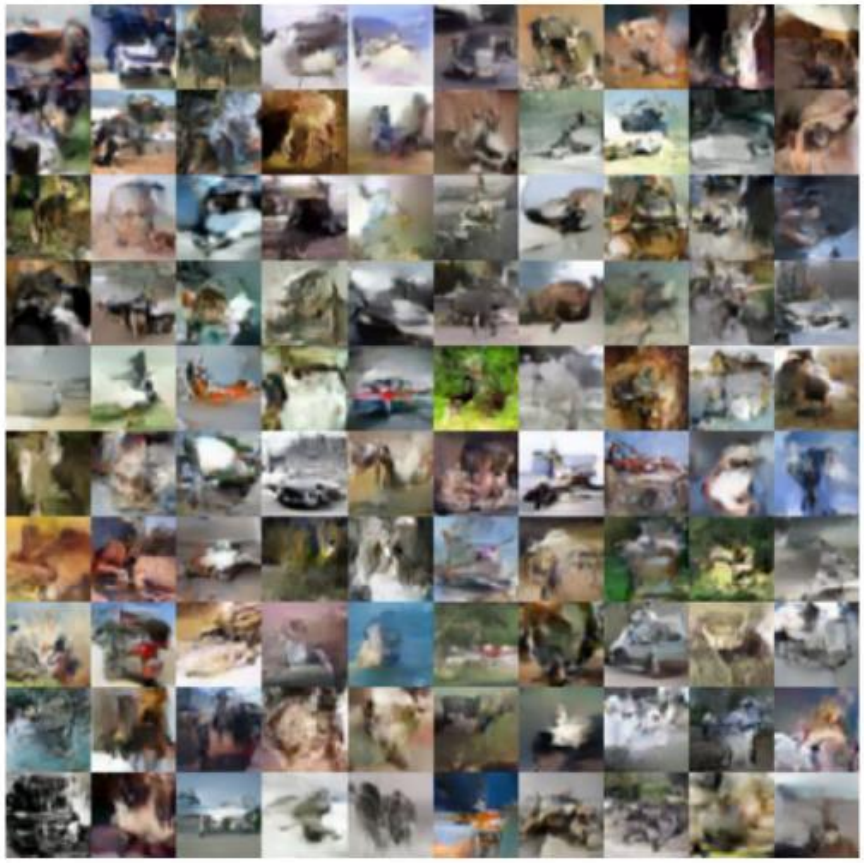
Sample Generation after Training

- **Generated samples w.r.t z**
  - Different dimensions of **z** encode interpretable factors of variation

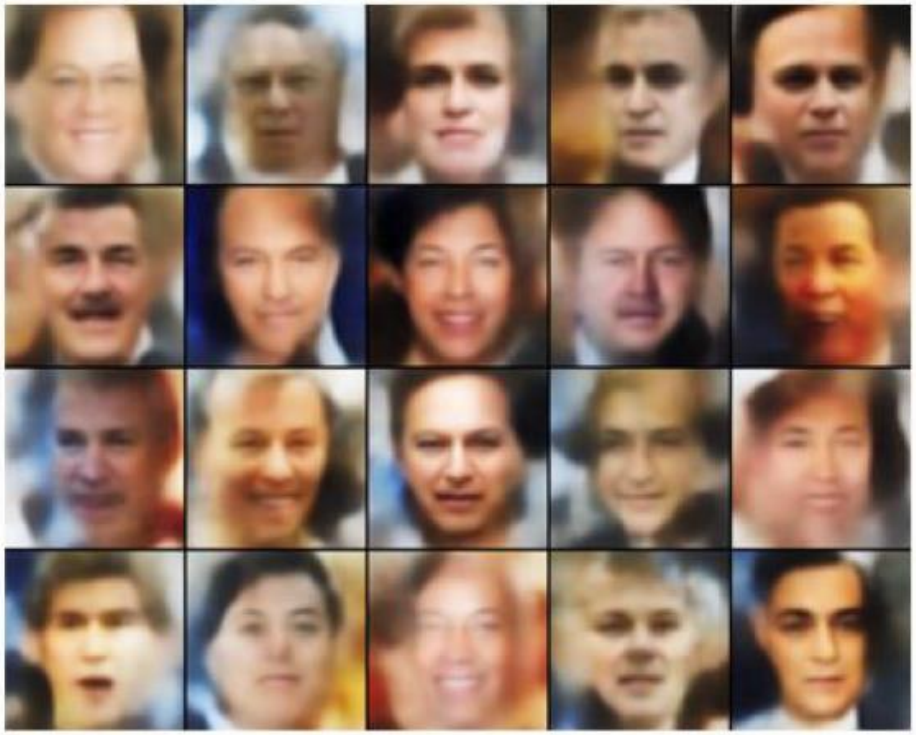


Sample Generation after Training

- **Generation quality**
  - Not very impressive, actually



32x32 CIFAR-10



Labeled Faces in the Wild

Summary

Variational autoencoder (VAE)

- Defines an intractable density ⇒ derive an ELBO and maximize it

$$\log p(x) \geq \underbrace{\mathrm{ELBO}(\theta, \phi) =}_{\substack{\theta\text{: parameters for decoder} \\ \phi\text{: parameters for encoder}}} \mathbb{E}_{z\sim q(z|x)}\underbrace{[\log p(x|z)]}_{\substack{\text{Expected log likelihood} \\ \text{of the data}}} - D_{KL}\underbrace{[q(z|x)||p(z)]}_{\substack{D_{KL}\text{ between} \\ \text{prior } p(z) \text{ and } q(z|x)}}$$

- Add "encoder" to learn parametric $q(z|x)$ that approximates $p(z|x)$

$$x \xrightarrow[\text{encoder}]{q(z|x)} z \xrightarrow[\text{decoder}]{p(x|z)} x$$

  - $p(x|z)$: now viewed as "decoder"
  - Encoder learns distribution parameters of $q(z|x)$
  - z is sampled from $q(z|x)$

- Trained VAE = maximized ELBO (tractable)

  = learn θ and ϕ by backprop (+reparameterization trick)

Thank You !